

Prosjektbeskrivelse – IT-utviklingsprosjekt (2IMI)

Prosjekttittel

Internet Boardgame Database (IBDb)

Bidragstere

Sivert M. Hansen (Individuelt prosjekt)

1. Prosjektidé og problemstilling

Beskrivelse

- Hva er prosjektet?

Prosjektet en Flask-applikasjon som jeg utvikler for å lære meg python-rammeverket og å utvikle en nettside med login og andre lignende funksjoner. Det startet som en skoleoppgave og vi har krav om blant annet å et inloggings-system på plass. Ellers vil jeg utforske funksjoner som søking, visning av database-innhold, administrering av brukere og brukerdashbord.

- Hvilket problem løser det?

Nettsiden skal la deg som sagt søke opp brettspill og finne info om de. Dette kan man bruke som et verktøy for å researche spillet eller si noe om hvilke målgrupper spillet passer. Da slipper du å komme til spillekvelden med et spill ingen liker!

- Hvorfor er løsningen nyttig?

Nettsiden fungerer som et oppslagsverk for brettspill. Det betyr at du kan søke opp spill du allerede har for kanskje å hvordan det spilles om du f.eks. mistet reglene. Dersom du ikke har et spill kan du bruke nettsiden til å lese deg opp på nett isteden for å måtte dra til byen eller kjøpesenteret.

Målgruppe

Hvem er løsningen laget for?

Løsningen er laget for de som spiller og liker brettspill, og de som vil finne ut mer om de. Det kan f.eks. være før de velger å kjøpe det, eller ikke, p.g.a. det de leste om det. Det kan også f.eks. være de som har mistet regler og glemte hva målet i spillet var som bruker siden for å minne seg selv på dette.

Plan for Eksamendagen -- ITK2004 5. juni 2026

Videre kan du lese om det jeg har planlagt å utføre på eksamensdagen, den tverrfaglige og praktiske eksamenen i IT-faget. Man vil også finne dette i kortere form på Kanbanboardet koblet til dette prosjektet.

Lage et ordentlig brukerdashbord

Denne er veldig omfattende så jeg deler den opp i mindre deler:

Endre egen brukerinfo

Hvis brukeren finner ut at de plutselig vil bruke en annen email, et annet brukernavn eller bytte passord, burde de kunne det. I brukerdashbordet vil jeg derfor legge til denne funksjonaliteten. Dette er relativt enkel ting som kan vise litt kompetanse innenfor Flask og SQL.

Gi admin og eller editor mer kraft

Til nå er den eneste forskjellen på en vanlig bruker og admin/editor muligheten til å registrere nye brettspill. I tillegg er det ingen praktiske forskjeller mellom admin og editor så langt. Jeg vil at admin skal kunne slette brukere og kunne endre informasjon som tilhører brettspillene. Da slipper jeg å gå inn i databasen for å gjøre dette manuelt. Her får jeg vise konstruksjon av en SQL-join foran eksaminator, eller lage den på forhånd for å se vise den frem og forklare hvordan den fungerer.

Annet

Egen side til hvert av brettspillene. Her skal blant annet beskrivelsen komme til bruk, noe den ikke egentlig har vært fra før.

Lage en side for personvernserklæring/TOS

Sier seg litt selv, bare en som forteller deg hvordan jeg bruker dataen din på nettsiden. Jeg kan da lage en footer med lenke til TOS-en. Dette er en mulighet på eksamen for å vise noe innenfor etikk, lovverk og yrkesutøvelse og dekker loven for retten til informasjon. Dette vil bli en kort side med noe tekst som er mest for å ha noe fysisk å gripe tak i når jeg forklarer ulike personvernlover for sensor og eksaminator.

Favorittbrettspill

Jeg vil utvikle en funksjon som lar en bruker trykke en "favoritt-knapp" på brettspillene-infokortene som legger brettspillet til i dine favoritter. Disse favorittene kan da vises på din egen brukerside.

Hvis vi skal se på det mer detaljert vil jeg legge til en ny "hjerne-knapp" på hver av brettspillene på forsiden som kan trykkes på for å legge til i databasen i en koblingstabell mellom brettspill og brukere. Trykker du på knappen igjen, skal den fjernes fra tabellen. Når det gjelder koblingstabellen, kan jeg lage den på eksamen for å vise kunnskap til eksaminator og sensor innenfor SQL. Her kan jeg i samme slengen vise frem diagrammet for å forklare hvorfor tabellen vil fungere. Det er fullt mulig og kanskje smartest om jeg lager denne hjerteknappen sin stil i forberedelsen, slik at jeg ikke må bruke tid på CSS på eksamen.

For å utføre dette i Python Flask kan jeg lage en route som har parameter som kan hentes fra knapp i templateen der brettspillene vises. Så bruke en sql-setning for å se om data finnes for å så legge den til hvis den ikke finnes fra før.

Dette er en av de mer avanserte tingene jeg kan utføre som vil vise mye kompetanse om jeg klarer å utføre det.

Oversiktlig søkeresultater

Nå hentes bare alt ut av databasen og vises i en parentes, en rå python-liste. Dette er ikke optimalt. For å gjøre det enklest mulig kan jeg ta utgangspunkt i "brettspillkortene" jeg har fra før av. Jeg ser på dette som litt en backup-ting å holde på med, et ekstra gjøremål for sikkerhets skyld.

Disponering av Tid

Først må jeg jo sette opp raspberry pi-en og sikre at koblingen fungerer, så at nettsiden kjører.

Når sensor kommer innom for første gang, må jeg vise frem og forklare ideen bak prosjektet. Jeg kan vise frem en logget inn admin-bruker for å navigere og vise alle sidene. Så vil jeg fortsette ved å si planen min og at jeg skal prøve å lage et favoritt-knapp-system og lage tabellen i databasen foran sensor for å vise kompetanse.

Tredje runde kan jeg vise mer utvikling ved å vise koden jeg har utviklet for favorittfunksjonen til nå, og forhåpentligvis kode litt foran sensor.

Den siste runden kan jeg bruke på TOS og forklare til sensoren om ulike lover og regler innenfor personvern og hvordan jeg tenker på sikkerhet i løsningen min. Hvis jeg skulle få ekstra tid har jeg noen backup-oppgaver å holde på med, som ryddig søkeresultat.

3. Teknologivalg

Programmeringsspråk

- Python

Rammeverk

- Flask

Database

- MariaDB

Andre Verktøy

- Waitress (WSGI)
- GitHub
- GitHub Projects (Kanban)

4. Datamodell

Programstruktur

Under ser du en oversikt av de funksjonelle delene som trengs i programmet:

```
boardgame-site/  
├─ app.py  
├─ templates/
```

```
| | base.html  
| | index.html  
| | register.html  
| | login.html  
| | dashboard.html  
| | register_boardgame.html  
| | results.html  
| static/  
| | media/  
| | | stylesheets/  
| | | style.css  
| | | faq.css  
| .env
```

Oversikt over tabeller

Tabell 1:

- Navn: user
- Beskrivelse: Innholder en brukers info om email, brukernavn og et hashet og saltet passord.

Tabell 2:

- Navn: boardgame
- Beskrivelse: Inneholder brettspillet navn, hvilket år det kom ut, de som lagde det, de som publiserte det og en beskrivelse.

Tabell 3:

- Navn: role
- Beskrivelse: Inneholder forskjellige roller som brukere kan ha. Her må jeg kjøre en manuell Insert, eller lage en funksjon i Python-filen.

Tabell 4:

- Navn: question
- Beskrivelse: Inneholder spørsmål om nettsiden fra brukere. Den har derfor fremmednøkkel til brukere sin ID.

Tabellstruktur i Databasen

Videre ser du strukturen på kommandoene brukt til å skape tabellene. Hvis du vil ha en mer grafisk fremstilling av tabellene, kan du sjekke ut [tabellstruktur.md](#) som du finner i dokumentasjonsmappen.

```
-- Rolletabell (nr. 3)  
CREATE TABLE role (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(20)  
);  
-- Innhold til rolletabell  
INSERT INTO role (name) VALUES ("admin"), ("editor"), ("user");
```

```
-- Rolletabellens innhold ser dermed slik ut:
+----+-----+
| id | name  |
+----+-----+
|  1 | admin |
|  2 | editor|
|  3 | user  |
+----+-----+

-- Brukertabell (nr. 1)
CREATE TABLE `user` (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(255) NOT NULL UNIQUE,
  email VARCHAR(255) NOT NULL UNIQUE,
  password CHAR(60) NOT NULL,
  role_id INT, FOREIGN KEY (role_id) REFERENCES role(id) DEFAULT 3,
  active INT
);

-- Brettspilltabell (nr. 2)
CREATE TABLE boardgame (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  year_published INT,
  creator VARCHAR(255),
  publisher VARCHAR(255),
  img_filename VARCHAR(255),
  description TEXT CHARACTER SET utf8mb4
);

-- Spørsmålstabell (nr. 4)
CREATE TABLE question (
  id INT AUTO_INCREMENT PRIMARY KEY,
  question TEXT NOT NULL,
  created_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  user_id INT NOT NULL,
  FOREIGN KEY (user_id) REFERENCES `user`(id)
);
```

Hvordan sette opp dette systemet

Før du starter må du ha installert disse på systemet (dependencies):

- git
- python

Deretter kan du starte ved å kclone prosjektet:

```
git clone https://github.com/sivertmh/boardgame-site.git
```

Så må du installere nødvendige pakker for appen (Det er lurt å gjøre dette i et **venv** i python).

Hvis du vil opprette et venv:

```
# Om du ikke har et venv fra før  
python -m venv .venv
```

Så last ned pakkene i ved hjelp av requirements-filen:

```
pip install -r requirements.txt
```

For å få kobling til database, må du overføre dotenv-filen manuelt, siden den ikke ligger på Github (her er jeg i samme mappe som .env):

```
sudo scp .env [user]@[host]:[sti_til_destinasjon]
```

Nå kan du kjøre prosjektet lokalt med Flask fra terminalen:

```
python -m flask run
```

Du kan også kjøre med Waitress (kan da nås av andre på LAN):

```
# 0.0.0.0 gjør den tilgjengelig utover LAN-et med ip-adressen til systemet den  
kjøres på  
waitress-serve --host 0.0.0.0 app:app  
  
# Eller, mer eksplisitt:  
waitress-serve --listen 0.0.0.0:8080 app:app
```

Utenom server/database, er dette alt du trenger for grunnleggende bruk/test av Flask-appen. Uten kobling til database vises ikke brettspill og login vil ikke fungere. Hvis du endrer databasekoblingen til en db du har tilgang til, vil du kunne kjøre *app.py* og tabeller vil opprettes.

Hvis du får tilgang til database-filen kan du kjøre denne kommandoen for å importere databasen inn i Mariadb (Windows):

```
Get-Content "[databasefil]" | mariadb -u [brukernavn] -p [database]
```

Tilsvarende på linux vil være mye enklere:

```
mariadb -u [brukernavn] -p [database] < [databasefil]
```

Kilder:

Du finner kilder i dokumentet [klidelite.md](#) som er i mappen *dokumentasjon*.